HEAD Bugs: Annotator-Judged Human-Easy but AI-Difficult Debugging Tasks

Yiting Dai **Melvyn Tan** **Tej Gaonkar** **Qingran Wu** **Teng Hou**

Abstract

We study a model-relative regime of debugging tasks that annotators judged easy for a competent programmer but that remain difficult for code-repair agents. Using a two-layer operational definition, we first separate tasks by annotation-based human difficulty and then isolate a canonical HEAD-20 subset inside the annotator-judged human-easy slice via five-seed Mistral Small `act_only` performance. HEAD is supported by five findings. First, the two-layer split exposes a non-trivial human-easy left tail: among 44 annotator-judged human-easy tasks, 20 remain HEAD under the weak-model baseline. Second, the human-difficulty layer is not a human study but a manual rubric over 90 tasks that distinguishes recognizable local bug patterns from tasks requiring deeper algorithmic or stateful reasoning. Third, hint experiments show that localization information partly restores weak-model performance, but the gains are concentrated in QuixBugs, indicating that repair generation remains the broader bottleneck. Fourth, matched protocol–strategy reruns show that plain protocol switching is not enough on HEAD; the best Mistral conditions are `react + early_stop_restart` and `reflexion + self_consistency`, both at 45%. Fifth, after fixing tool-call compatibility, an independent `Llama-4-Scout` recheck still solves only 8/20 tasks, supporting limited cross-model transfer of the subset.

1 Introduction

Modern code agents can solve many benchmark tasks, yet they still fail on bugs that annotators would judge straightforward for a competent programmer. This mismatch suggests that model capability does not align cleanly with annotation-based notions of bug difficulty. Rather than treating all failures as belonging to one hard bucket, we ask whether there exists a structured regime of tasks that are annotator-judged human-easy but disproportionately difficult for agents.

We study that regime under the name **HEAD**: *Human-Easy but AI-Difficult*. Throughout the paper, "human-easy" is shorthand for an *annotator-judged* label derived from a manual rubric rather than from a direct human-subject experiment. The key idea is that HEAD is not a claim about intrinsic bug hardness. Instead, it is an operational category defined relative to a debugging setup: an annotator-judged human-easy task counts as HEAD when a weak debugging agent fails on it consistently under a canonical baseline. This lets us separate *definition* from *validation*. Mistral Small under `act_only` provides the definition baseline, while additional experiments probe whether the same subset remains difficult under other prompts, strategies, and model families.

The contribution is empirical and restrained:

1. a two-layer bug classification that isolates a 20-task canonical HEAD subset from an annotator-judged human-easy slice;
2. an `act_only` hint study showing that localization cues have uneven effects and do not fully explain HEAD;

Preprint.

3. a matched protocol–strategy study showing that trajectory control matters more than plain protocol switching on Mistral HEAD20;
4. a Reflexion ablation showing no stable aggregate gain over ReAct in the matched batch;
5. an independent `Llama-4-Scout` recheck showing that the Mistral-defined subset remains difficult for another model family.

2 Related Work

Recent work has established strong baselines for agentic debugging and code repair, but it has mostly emphasized aggregate benchmark performance. ReAct introduced the standard reasoning-plus-action pattern for tool-using agents [Yao et al., 2023], and Reflexion extended it with multi-episode self-critique and retry [Shinn et al., 2023]. SWE-agent studies end-to-end issue resolution in larger software environments [Yang et al., 2024], SWE-bench+ revisits the quality of coding benchmarks for LLM evaluation [Aleithan et al., 2024], and DebugBench targets debugging tasks with explicit bug categories such as syntax, logic, and reference errors [Tian et al., 2024]. Taken together, these papers show that agent performance is sensitive to benchmark design, tool access, and prompting, but they usually compare whole datasets rather than structured slices within them.

A second line of work studies inference-time control. ReAct and Reflexion are the clearest protocol baselines used here, but the broader question is how much performance depends on run-time structure rather than on the underlying model alone. The matched protocol–strategy study in our work fits this line directly: early stopping, restart, and self-consistency all reallocate finite debugging budget across trajectories instead of changing the task distribution itself.

The main difference from prior work is therefore not another benchmark ranking, but a different empirical object. Our work first isolates an annotator-judged human-easy region and then defines a model-relative hard subset inside it. Instead of asking which model is strongest on average, it asks which tasks remain difficult precisely where local human fixability would suggest they should be easy. This yields a more targeted view of cross-model transfer, localization sensitivity, and convergence failure than aggregate benchmark scores alone.

3 Task Construction and Experimental Setup

3.1 Benchmarks

The study contains 90 debugging tasks drawn from three sources:

- **QuixBugs**: 40 algorithmic program-repair tasks;
- **Mini-nightmare**: 10 compact, realistic debugging tasks;
- **DebugBench**: 40 debugging tasks spanning reference, syntax, logic, and multiple-error patterns [Tian et al., 2024].

The experiments use a single local debugging-agent setup across all reported conditions so that task difficulty, prompting, and model family can be compared under a shared environment.

3.2 Two-Layer Difficulty Definition

The first layer separates tasks by human difficulty using manual task annotations over all 90 tasks. This layer is annotation-based rather than experimental: it should be read as an annotator judgment about likely human repair difficulty, not as a measured human-subject result. The annotation rubric is intentionally operational rather than psychological. A task is labeled `human-easy` when a competent programmer can plausibly find and repair the bug by recognizing a familiar error pattern directly from the code and test output, without reconstructing the surrounding algorithm in depth or tracing a long chain of state changes. Typical cases include syntax errors, typos, wrong variable or attribute names, obvious operator mistakes, simple off-by-one errors whose direction is revealed by the failing output, and missing returns or guard conditions that are already implied by the function contract.

A task is labeled `human-difficult` when repairing it plausibly requires deeper algorithmic understanding, extended control-flow tracing, domain-specific reasoning, concurrency or mutable-state

Table 1: Experimental design summarized by role rather than by chronological run order.

Layer	Scope	Models / conditions	Purpose
Human-difficulty annotation	90 tasks	manual rubric	separate annotator-judged human-easy from human-difficult
AI-difficulty definition	44 annotator-judged human-easy tasks	Mistral Small, `act_only`, 5 seeds	define Easy / Intermediate / HEAD
Hint study	HEAD20	two Mistral `act_only` reruns and one 120B `act_only` rerun at L0/L2/L4	test localization sensitivity within the definition protocol
Matched protocol study	HEAD20	3 protocols $\times$ 4 strategies, shared budgets	compare trajectory-control interventions
Mechanism ablation	HEAD20	ReAct / Reflexion matched reruns	test whether Episode 2 drives gains
Cross-family recheck	HEAD20	Llama-4-Scout, `act_only` baseline	probe transfer beyond Mistral

Table 2: Comparability of the reported experiment families. Numerical comparisons are intended only within the directly comparable groups.

Experiment family	Scope	Shared settings inside family	Direct comparison policy
Definition batch	44 annotator-judged human-easy tasks	Mistral `act_only`, 5 seeds	compare only the second-layer counts and membership
Hint reruns	HEAD20, localization interventions	independent `act_only` reruns, seed 42, 25 turns, 50k tokens	compare hint levels within a rerun; compare reruns qualitatively
Matched protocol batch	HEAD20, 12 protocol–strategy conditions plus baseline trajectories	Mistral, seed 42, 25 turns, 50k tokens	compare Tables 7, 8, and 9 within batch
Matched ablation batch	HEAD20, ReAct/Reflexion mechanism study	Mistral, seed 42, 25 turns, 50k tokens	compare conditions within Table 10 only
Scout recheck batch	HEAD20, external model-family check	Scout `act_only` baseline, post-fix parser	compare only within the Scout recheck table

analysis, or untangling multiple interacting errors. The annotation records not only the binary label but also a short reasoning field, a coarse bug-pattern description, and the approximate fix region. The purpose of these annotations is to anchor the first layer in explicit repair criteria before any model outcomes are considered.

The second layer refines only the annotator-judged `human-easy` slice using five-seed Mistral Small `act_only` pass counts:

- **Easy**: at least 3/5 passes;
- **Intermediate**: exactly 2/5 passes;
- **HEAD**: at most 1/5 passes.

Table 3: Two-layer classification used in this study. "Human-easy" and "human-difficult" denote annotation labels, not direct human-subject measurements.

Slice	Count
`human-easy`	44
`human-difficult`	46
`human-easy / Easy`	16
`human-easy / Intermediate`	8
`human-easy / HEAD`	20

Table 4: Composition of the canonical HEAD20 subset by benchmark family.

Family	Count
DebugBench	7
Mini-nightmare	4
QuixBugs	9

This design makes HEAD a high-precision operational subset rather than a universal theory of AI difficulty.

3.3 Models and Conditions

The model roles are:

- **Definition model**: `api-mistral-small-3.2-2506`;
- **Strong reference**: `api-gpt-oss-120b` as a strong-model comparison in the hint study, using the same `act_only` protocol at L0/L2/L4;
- **Independent recheck**: `api-llama-4-scout`.

The main protocol families are `act_only`, `react`, and `reflexion`. The matched Mistral rerun compares 12 protocol–strategy conditions under a shared seed, token budget, and turn budget.

4 Main Results

The results are organized as a progression rather than as a flat list. We first establish that HEAD is a real slice inside the annotator-judged human-easy region and then strengthen that definition with an external recheck on a different model family. We next ask what kind of difficulty this slice reflects, using hints and trajectory aggregates as diagnosis tools. Finally, we test which interventions help once the slice has been isolated. Table 2 should be read alongside this section: several tables come from different rerun batches, so only the within-batch comparisons are intended as strict numerical comparisons.

4.1 Existence: HEAD forms a real left tail inside annotator-judged human-easy tasks

Table 3 shows the first main result: annotator-judged human-easy does not imply agent-easy. Among 44 tasks judged human-easy by the rubric, only 16 are easy for the Mistral baseline, 8 lie in an intermediate band, and 20 fall into the HEAD subset. This means nearly half of the annotated human-easy slice still looks difficult to the weak agent under the canonical setup.

This result is important because it separates HEAD from generic benchmark hardness. HEAD is not built from human-difficult tasks. It is the left tail *inside* the annotator-judged human-easy region. Table 4 further shows that the canonical subset is not tied to a single benchmark source: it contains 7 DebugBench tasks, 4 Mini-nightmare tasks, and 9 QuixBugs tasks. The phenomenon therefore survives across both algorithmic repair tasks and compact debugging tasks with more open-ended execution traces.

Table 5: Independent Llama-4-Scout recheck on HEAD20.

Family	Pass rate	Avg. tokens
Overall	8/20 (40.0%)	36,260
DebugBench	3/7 (42.9%)	35,038
Mini-nightmare	0/4 (0.0%)	53,540
QuixBugs	5/9 (55.6%)	29,531

Table 6: `Act_only` hint response on the canonical HEAD20 subset across complete runs (all with seed 42, 25 turns, 50k tokens).

Hint level	Mistral run	Mistral rerun	120B
L0 (no hint)	8/20 (40%)	5/20 (25%)	19/20 (95%)
L2 (function name)	6/20 (30%)	6/20 (30%)	18/20 (90%)
L4 (line-level hint)	9/20 (45%)	6/20 (30%)	20/20 (100%)

4.2 External recheck: the Mistral-defined subset remains difficult for Scout

The definition itself is Mistral-relative, so it is important to ask whether the resulting subset is still difficult for another model family. We therefore run an independent recheck with `api-llama-4-scout`. After extending the runner to accept the model's bracketed pseudo-tool-call format, Scout solves only 8/20 tasks under `act_only + baseline`.

This recheck does not redefine HEAD; Mistral remains the definition model. It does, however, strengthen the interpretation of HEAD as a model-relative regime rather than a one-model accident. In particular, the complete failure on Mini-nightmare suggests that the hardest part of HEAD20 is not just an artifact of one model API or one prompt family. Because the Scout evidence is a single post-fix rerun, it should be read as external support rather than as a full stability study. The task-level list in the appendix is consistent with the family totals in Table 5: five of the eight Scout successes come from QuixBugs, three come from DebugBench, and none come from Mini-nightmare.

4.3 Diagnosis: HEAD combines localization sensitivity with a convergence bottleneck

4.3.1 Hints help unevenly, even within the definition protocol

The hint study keeps the definition protocol fixed and varies only localization information inside `act_only`. Table 6 reports two complete Mistral runs and one complete 120B strong-reference run over all 20 HEAD tasks at levels L0/L2/L4.

These hint runs are separate from the matched protocol batch used in Tables 7, 8, and 9, so their L0 values should be read only as baselines *within the hint runs*, not as replacements for the 5/20 `act_only` baseline reported later. The main pattern is not a stable monotonic aggregate gain for Mistral. Across the run and rerun, Mistral varies from 8/20 to 5/20 at L0, stays at 6/20 for L2, and ranges from 9/20 to 6/20 at L4. The more robust pattern is family asymmetry: Appendix Tables 12 and 13 show that whatever gains appear are concentrated in QuixBugs, while DebugBench and Mini-nightmare remain largely resistant. In the original run, QuixBugs improves from 5/9 to 6/9 to 8/9; in the rerun, it moves from 5/9 to 6/9 to 6/9; DebugBench and Mini-nightmare are at or near zero throughout the rerun. This makes the diagnosis sharper. Localization cues can help a subset of algorithmic repair tasks, but they do not behave like a uniform rescue mechanism across HEAD20.

The strong-model run shows a different regime. 120B is already near saturation at L0 with 19/20 and reaches 20/20 at L4, while the family-level appendix table shows only small residual variation. In this regime, additional localization information behaves more like context sharpening than like rescue. Taken together, the runs support a narrower claim than the original single-batch hint table: localization sensitivity is real but benchmark-dependent, and for the weak model it is not a sufficient explanation of HEAD.

Table 7: Mistral baseline trajectory metrics on HEAD20 within the matched rerun batch.

Protocol	Pass	First edit	First fix	Edit→fix gap	Patch attempts	Test runs
`act_only`	5/20	5.0	8.2	2.8	4.0	4.0
`react`	7/20	5.0	6.9	1.0	4.5	3.4
`reflexion`	6/20	5.2	5.7	1.0	5.1	3.4

Table 8: Best-performing Mistral conditions on HEAD20 from a single matched rerun batch (seed 42, 25-turn, 50k-token budget).

Condition	Pass rate	Avg. tokens
`react + early_stop_restart`	9/20 (45%)	36,770
`reflexion + self_consistency`	9/20 (45%)	37,288
`reflexion + early_stop_restart`	8/20 (40%)	39,143
`react + self_consistency`	8/20 (40%)	42,329
`act_only + baseline`	5/20 (25%)	41,638

4.3.2 Trajectory aggregates indicate failure to converge rather than failure to start

Table 7 is computed from the same matched rerun batch used later in Tables 8 and 9, so the baseline values are now directly comparable across Section 3.4. All 20 baseline runs in each protocol reach both editing and testing, and the first edit appears at roughly turn 5 regardless of protocol. The columns are not all averaged over the same subset: *First edit* is averaged over all 20 runs in a protocol, while *First fix* and *Edit→fix gap* are averaged only over successful runs. This makes it difficult to describe HEAD as a failure to begin acting. Instead, the main variation lies in whether those attempts converge: `act_only` reaches 5/20, `react` reaches 7/20, and `reflexion` reaches 6/20, even though all three protocols are already editing and testing consistently.

These aggregates therefore suggest that HEAD is not mainly a regime where the weak model fails to locate a starting point. Instead, the agent often reaches an editable region, begins modifying code, and still fails to convert those attempts into a correct fix. Successful runs that do converge do so relatively soon after the first edit, with average edit-to-fix gaps of 2.8 turns for `act_only` and 1.0 turn for both `react` and `reflexion`. The more persistent difficulty is converting engagement into correct repairs at the task level. This observation complements the act-only hint reruns: even when line-level hints help on some QuixBugs tasks, localization cues alone do not produce a stable broad recovery across HEAD20 for the weak model.

4.4 Intervention: protocol–strategy combinations help by improving convergence, not by adding protocol complexity alone

Section 3.3.2 showed that HEAD is not well described as a failure to begin acting. The weak model usually reaches an editable region and starts making changes, but then fails to convert those attempts into a correct fix. The next question is therefore not simply which protocol is "stronger," but which intervention most effectively improves convergence once a run is already underway. The evidence in this section points to a consistent answer: the best gains come from protocol–strategy combinations that restructure how search unfolds, especially by shortening unproductive trajectories or redistributing budget across independent attempts.

4.4.1 Protocol switching alone is not enough on HEAD

The matched Mistral rerun compares 12 protocol–strategy conditions on HEAD20 in a single directly comparable batch. The key takeaway is that the best gains do not come from plain protocol upgrades by themselves. They come from protocol–strategy combinations that reduce the cost of staying on a bad trajectory.

Two observations follow from Tables 8 and 9. First, plain protocol switching is helpful but limited within this matched batch: the baseline progression from `act_only` to `react` to `reflexion` is only 25% → 35% → 30%. Second, the largest gains appear only when a protocol is paired with a

Table 9: Improvement over the same-protocol baseline within the matched rerun batch.

Protocol / strategy	Pass rate	Delta vs. baseline
act_only + baseline	5/20 (25%)	—
act_only + checklist	7/20 (35%)	+10 points
act_only + early_stop_restart	7/20 (35%)	+10 points
react + baseline	7/20 (35%)	—
react + self_consistency	8/20 (40%)	+5 points
react + early_stop_restart	9/20 (45%)	+10 points
reflexion + baseline	6/20 (30%)	—
reflexion + early_stop_restart	8/20 (40%)	+10 points
reflexion + self_consistency	9/20 (45%)	+15 points

Table 10: Reflexion mechanism ablation on HEAD20 from a separate matched ablation batch (seed 42, 25-turn, 50k-token budget).

Condition	Pass rate	Avg. tokens
react_baseline	8/20 (40%)	38,306
reflexion_baseline	8/20 (40%)	37,230
reflexion_ep1_only	7/20 (35%)	37,813
restart_without_reflection	5/20 (25%)	46,284

particular strategy. Within-protocol deltas are consistently larger than the baseline protocol gaps: `react + early_stop_restart` improves over `react + baseline` by 10 points, and `reflexion + self_consistency` improves over `reflexion + baseline` by 15 points. Relative to the convergence diagnosis in Section 3.3.2, this pattern is informative: the winning interventions are precisely the ones that reduce the cost of staying on a bad path, either by terminating it earlier or by allocating budget across multiple shorter attempts.

The broader ranking in Appendix Table 15 reinforces this interpretation. Plain baseline ordering is modestly separated—`act_only` at 25%, `reflexion` at 30%, and `react` at 35%—but none of these baseline moves is decisive on its own. The strongest conditions are the paired interventions that either cut off unproductive trajectories early (`early_stop_restart`) or allocate budget across multiple independent attempts (`self_consistency`). The bottom of the ranking is also informative: `reflexion + checklist` falls to 20% despite using the heaviest protocol, which argues against a simple "more reasoning is better" interpretation. On this subset, weak-model performance depends more on how the run budget is structured than on protocol label alone.

4.4.2 Reflexion does not show a stable aggregate edge in the matched ablation

We also include a matched Reflexion mechanism ablation on HEAD20. All conditions in Table 10 were rerun in one batch to avoid comparing a new ablation to older baselines. This table is internally comparable, but it is still a separate batch from Tables 8 and 9. Accordingly, the 8/20 baseline numbers here should be read as ablation-batch baselines rather than as replacements for the 35% and 30% matched-rerun baselines above.

The ablation does not reproduce a clear aggregate Reflexion-over-ReAct advantage. Reflexion baseline ties ReAct baseline at 8/20, and the family breakdown is also identical at 1/7 DebugBench, 0/4 Mini-nightmare, and 7/9 QuixBugs. The two conditions exchange only one QuixBugs success in either direction. Likewise, `reflexion_ep1_only` reaches 7/20, only one task behind ReAct, so ReAct does not behave as a strict lower bound on Reflexion Episode 1. Most importantly, neither two-episode condition produces an `Ep1 fail` $\rightarrow$ `Ep2 pass` rescue. Meanwhile, clean restart without reflection is both weaker and more expensive. The most conservative interpretation is therefore negative: in the current matched batch, Episode 2 is not a demonstrated positive contributor, and naive restart alone is not the source of the gains seen in Table 9.

This is substantively different from saying that Reflexion never helps. The narrower claim supported here is that the observed gains on HEAD20 are better explained by convergence management than by a stable second-episode rescue mechanism. Appendix Table 16 and its accompanying readouts show the same pattern at slightly finer granularity: no Episode-2 rescues are observed, and `restart_without_reflection` drops to 25% while consuming the most tokens. The evidence therefore supports a specific reading of Section 3.3 as a whole. On HEAD, the useful interventions are not simply those that add another reasoning stage; they are the ones that prevent the agent from spending too much of its budget on a single failing trajectory.

5 Discussion

The results support a narrower interpretation of HEAD than a generic "hard bug" label. HEAD is a model-relative regime inside the annotator-judged human-easy slice: the Mistral-based definition isolates failures that are not trivially explained by the first-layer rubric, and the Scout recheck shows that this subset transfers non-trivially to another model family. At the same time, the evidence does not support a pure localization story. The act-only hint runs are uneven and batch-sensitive, with stable gains only on QuixBugs, while the matched trajectory aggregates show that all three Mistral baselines usually reach editing and testing. The most coherent reading is therefore that many HEAD failures arise after engagement, during convergence toward a correct repair.

The intervention results sharpen that interpretation. Baseline protocol gaps are modest, Reflexion baseline does not separate from ReAct in the matched ablation, and naive restart is both weaker and more expensive. The strongest gains come instead from protocol–strategy combinations such as `react + early_stop_restart` and `reflexion + self_consistency`, which limit how much budget is spent on a single failing trajectory. This supports a bounded claim: HEAD is a meaningful and partially transferable regime, and its failures are more consistent with post-edit convergence problems than with failure to start, but the current evidence is still aggregate rather than a direct trajectory-level mechanism proof.

6 Limitations and Future Work

- The first layer is annotation-based rather than a direct human-subject study, so "human-easy" should be read as a rubric-derived label rather than as measured human repair performance.
- The canonical HEAD definition is tied to one weak model family and one protocol baseline.
- Some results, especially the Scout recheck, are currently single-seed.
- The current evidence is mostly aggregate; it shows *that* HEAD exists, but not yet a full trajectory-level account of *why* each subtype fails.

The next steps are multi-seed cross-model validation, subtype-level analysis inside HEAD20, and manual trajectory annotation that distinguishes localization, repair, and search-control failures.

7 Conclusion

This study isolates a 20-task subset of annotator-judged human-easy bugs that remains difficult for a weak debugging agent under a fixed baseline. The evidence supports three claims: the subset is a real left tail inside the annotated human-easy slice, its difficulty transfers non-trivially to Scout, and its failures are better explained by uneven localization sensitivity plus post-edit convergence problems than by failure to start. The strongest improvements come from protocol–strategy combinations that manage bad trajectories rather than from protocol complexity alone.

Appendix

A Two-Layer Definition Details

The canonical HEAD definition uses five-seed Mistral Small `act_only` performance inside the `human-easy` slice. The thresholds are repeated here for completeness.

Table 11: Operational definition of the second-layer categories inside `human-easy`.

Category	Rule
`Easy`	at least 3/5 Mistral `act_only` passes
`Intermediate`	exactly 2/5 passes
`HEAD`	at most 1/5 passes

This definition is narrow on purpose. It is intended to produce a stable high-precision subset rather than a universal definition of AI debugging difficulty.

B Hint Study Details

Table 12: Family-level Mistral hint response on HEAD20 under `act_only` (run).

Family	L0	L2	L4
DebugBench	2/7	0/7	1/7
Mini-nightmare	1/4	0/4	0/4
QuixBugs	5/9	6/9	8/9

Table 13: Family-level Mistral hint response on HEAD20 under `act_only` (rerun).

Family	L0	L2	L4
DebugBench	0/7	0/7	0/7
Mini-nightmare	0/4	0/4	0/4
QuixBugs	5/9	6/9	6/9

Across the Mistral run and rerun, QuixBugs remains the only family with consistent hint sensitivity. 120B is near-saturated across all three families, with only small residual variation at L0 and L2.

C Matched Mistral Protocol–Strategy Table

Mini-nightmare is omitted from the last two columns because it is almost entirely zero in this matched rerun; only `act_only + checklist` solves one of the four tasks. Accordingly, the displayed family counts do not sum to the overall totals in the first two columns. Most of the variation comes from QuixBugs and, to a lesser extent, DebugBench.

D Reflexion Mechanism Details

Additional readouts:

- Episode-1 pass rate: 7/20 (35%).
- Episode-2 rescues in baseline: 0.
- Episode-2 rescues in restart-without-reflection: 0.
- Baseline success but restart-without-reflection fail: 5 tasks.
- Baseline success but ReAct fail: 1 task.

Table 14: Family-level 120B hint response on HEAD20 under `act_only`.

Family	L0	L2	L4
DebugBench	7/7	6/7	7/7
Mini-nightmare	3/4	4/4	4/4
QuixBugs	9/9	8/9	9/9

Table 15: Full matched Mistral protocol–strategy ranking on HEAD20.

Protocol	Strategy	Pass	Pass %	Avg. tokens	DebugBench	QuixBugs
`react`	`early_stop_restart`	9/20	45.0	36,770	2/7	7/9
`reflexion`	`self_consistency`	9/20	45.0	37,288	2/7	7/9
`reflexion`	`early_stop_restart`	8/20	40.0	39,143	1/7	7/9
`react`	`self_consistency`	8/20	40.0	42,329	2/7	6/9
`act_only`	`checklist`	7/20	35.0	38,778	0/7	6/9
`react`	`baseline`	7/20	35.0	39,141	1/7	6/9
`act_only`	`early_stop_restart`	7/20	35.0	41,825	2/7	5/9
`reflexion`	`baseline`	6/20	30.0	39,582	1/7	5/9
`act_only`	`baseline`	5/20	25.0	41,638	0/7	5/9
`react`	`checklist`	5/20	25.0	44,034	0/7	5/9
`act_only`	`self_consistency`	5/20	25.0	45,586	0/7	5/9
`reflexion`	`checklist`	4/20	20.0	45,406	0/7	4/9

- ReAct success but baseline fail: 1 task.

No matched condition in this ablation shows a stable Episode-2 rescue effect.

E Independent Scout Recheck Details

E.1 Compatibility Note

Scout initially emitted bracketed pseudo-tool calls such as `[run_tests()]` rather than formal API tool calls. The fallback parser was then extended to accept this format, and only the post-fix rerun is treated as valid evidence.

E.2 Task-Level Summary

The eight successful Scout tasks are:

- `debugbench_012_corporate_flight_bookings`
- `debugbench_031_next_greater_element_i`
- `debugbench_037_minimum_bit_flips_to_convert_number`
- `quixbugs_is_valid_parenthesization`
- `quixbugs_mergesort`
- `quixbugs_quicksort`
- `quixbugs_to_base`
- `quixbugs_wrap`

Scout solves 0/4 Mini-nightmare tasks even after the tool-call fix.

References

Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. SWE-Bench+: Enhanced coding benchmark for llms. *arXiv preprint arXiv:2410.06992*, 2024.

Table 16: Detailed Reflexion mechanism ablation.

Condition	Pass	Avg. tokens
`react_baseline`	8/20	38,306
`reflexion_baseline`	8/20	37,230
`reflexion_ep1_only`	7/20	37,813
`restart_without_reflection`	5/20	46,284

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Yinxu Pan, Yesai Wu, Hui Haotian, Liu Weichuan, Zhiyuan Liu, and Maosong Sun. Debugbench: Evaluating debugging capability of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.